

BURSA TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

BLM0121 – Nesneye Yönelik Programlama (Şube1)
Yılıçi Sınavı

Ad&Soyad	: CEVAP ANAHTARI
Öğrenci Numarası	:

Akademik yıl : 2025-2026
Dönem : Bahar
Tarih : 25 Nisan 2026 – 09:00
Sınav süresi : 90 dakika
Öğr. Görevlisi : Doç. Dr. Ergün GÜMÜŞ

Soru	1	2	3	4	5	Toplam
Puan	28	20	20	12	20	100
Not						

KURALLAR

- Sınava başlamadan önce Ad&Soyad ve Öğrenci numarası alanlarını doldurunuz.
- Sınav öncesinde ve süresince sınav gözetmenlerinin tüm uyarılarına uymanız gerekmektedir.
- Sınav öncesinde cep telefonlarınızı KAPATINIZ!
- Sınav boyunca masanızın üzerinde bulunabilecek malzemeler sadece sınav kâğıdınız, kalem ve silgidir.
- Sınav süresince herhangi bir nedenle birbirinizle konuşmak ve malzeme (silgi, kalem, kâğıt vb.) alışverişi yasaktır.
- Bu kuralların herhangi birine uymamak kopya çekmeye yönelik bir hareket olarak değerlendirilir ve ilgili makamlara bildirilir.

1) Aşağıda, “KitapYonetimSistemi” isimli bir Java sınıfının kısmi kodu verilmiştir. Bu sistem, kitapları (bir de “Kitap” sınıfımız var) bir dizi içinde tutar. Dizi boyutu başlangıçta 2’dir, ancak yeni kitap eklendiğinde dizi dolarsa dizi boyutu iki katına çıkar. Ayrıca kitap silme işlemi vardır (silinen yere null konur). Sizden istenen, a, b, c, d etiketli bölümlere uygun kod parçalarını yazmanızdır.

```
public class Kitap {
    private int id;
    private String baslik;
    public Kitap(int id, String baslik) { this.id = id; this.baslik = baslik; }
    public int getId() { return id; }
    public String getBaslik() { return baslik; }
    public String toString() { return id + ": " + baslik; }
}

public class KitapYonetimSistemi {
    private Kitap[] kitaplar;
    private int sayac; // mevcut kitap sayısı

    public KitapYonetimSistemi() {
        kitaplar = new Kitap[2];
        sayac = 0;
    }

    // ---- (a) Kitap ekleme metodu ----
    public void kitapEkle(Kitap k) {
        //Eklemeden sonra dizi doluyorsa dizi boyutunu 2 katına çıkarın. Sayacı artırmayı unutmayın.
    }

    // ---- (b) Diziyi büyütme metodu ----
    private void diziyiBuyut() {
        //Dizi boyutunu 2 katına çıkartan kodu yazın.
    }

    // ---- (c) ID'ye göre kitap silme ----
    public int kitapSil(int id) {
        //İlgili id'deki kitabı bulup yerine null koyun. Sayacı azaltmayı unutmayın.
        //Ayrıca, silinen elemanın dizideki indeksini geriye döndürün. id geçersiz ise -1 döndürün.
    }

    // ---- (d) Tüm kitapları ekrana yazdırma ----
    public void listele() {
        // null olmayan elemanları yazdıran kodu yazın. toString() metodundan faydalanın.
    }
}
```

2) Aşağıdaki UML diyagramında “Ogrenci” sınıfı ve yanında bu sınıftaki her bir metodun yaptığı iş verilmiştir. Bu sınıfın Java kodunu yazınız.

Ogrenci
- ogrenciNo: int - ad: String - notlar: int[] - notSayisi: int
+ Ogrenci(ogrenciNo: int, ad: String) + notEkle(not: int): boolean + ortalamaHesapla(): double + getOgrenciNo(): int + getAd(): String + enYuksekNot(): int

notEkle(not): Eğer not sayısı 5’ten azsa notu diziyi ekler, notSayisi artırılır, true döner. 5’ten çoksa eklemeyiz, false döner

ortalamaHesapla(): Eklenen notların ortalamasını döndürür

getOgrenciNo(): getter metot

getAd(): getter metot

enYuksekNot(): Eklenen notlar içindeki maksimumu bulup döndürür

3) Aşağıdaki “Dikdortgen” sınıfında bazı hatalı ifadeler vardır. Her bir hatayı bularak hatalı satırın yanına doğrusunu yazınız.

```
public class Dikdortgen {
    private double en;
    private double boy;
    static int nesneSayisi;

    public Dikdortgen(double e) {
        en = e;
        boy = 1;    //boy = 1; varsayılan değer, burada hata yok
    }

    public Dikdortgen(double en, double boy) {
        boy = boy;           //this.boy = boy; olmalı
        this(en);            //bir üstteki satırda olmalı, yeri yanlış
        nesneSayisi++;
    }

    public double alanHesapla( ) {
        return en * boy;
    }

    public static double cevreHesapla( ) { //static metot sadece static değişkene erişir. imzadaki static
kelimesi silinmeli
        return 2 * (en + boy);
    }

    public static void getNesneSayisi( ) { //void olamaz. void yerine int yazılmalı
        return nesneSayisi;
    }
}

public class Test {
    public static void main(String[ ] args) {
        Dikdortgen d1 = new Dikdortgen(3, 4);
        Dikdortgen d2 = new Dikdortgen( ); //parametresiz konstrüktör yok. Üstteki sınıfa public
Dikdortgen(){ } konstrüktörü eklenmeli
        System.out.println("Alan: " + d1.alanHesapla( ));
        System.out.println("Çevre: " + d1.cevreHesapla( ));
        System.out.println("Toplam nesne: " + Dikdortgen.nesneSayisi);
    }
}
```

3. soruda bazı öğrenciler tek parametrelili public Dikdortgen(double e) {...} konstrüktöründe nesneSayisi değişkeninin değerinin artırılmadığını fark etmiştir. Bu cevap da hatalardan biri olarak kabul edilmiştir (opsiyonel cevap).

4) Aşağıda aşırı yüklenmiş üç adet “temizle” metodu bulunmaktadır:

- A) String temizle(String str);
- B) String temizle(String str, char c);
- C) String temizle(String str, boolean sadeceHarfler);

A türü “temizle”, parametre olarak aldığı bir metnin sadece başındaki ve sadece sonundaki boşlukları temizlenmiş bir versiyonunu döndürür.

B türü “temizle”, parametre olarak aldığı bir metnin içindeki belirli bir harfin (char c ile gösterilen) tüm tekrarlarının temizlendiği versiyonunu döndürür.

C türü “temizle” ise sadeceHarfler parametresi true ise A-Z ve a-z aralığındaki tüm harfleri, parametre false ise A-Z ve a-z aralığı dışındaki tüm harfleri metinden temizleyerek yeni bir metin döndürür.

İster her karakteri kontrol eden döngüler kurarak, isterseniz de String veya StringBuilder sınıfından bildiğiniz hazır metotları kullanarak bahsedilen üç “temizle” metodunu da yazınız.

5) Aşağıdaki ifadelerdeki boşlukları doğru şekilde doldurunuz.

- i. Java’da iki String’in içeriğini karşılaştırmak için metodu kullanılır.
- ii. Java’da bir sınıfın üye değişkenine sadece aynı paket içindeki sınıflar erişebiliyorsa, türünde erişim belirteci söz konusudur.
- iii. Bir dizinin elemanlarını sıralamak için java.util paketindeki sınıfının isimli metodu kullanılır.
- iv. Java’da her satırı farklı sayıda eleman içerebilen dizilere adı verilir.
- v. Değerleri yeniden kullanılabilirsin diye Java’daki String literal’ler’da tutulur.
- vi. Java’da bir dizinin uzunluğuna (eleman sayısına) ile erişilirken, String nesnesinin uzunluğuna metodu ile erişilir. Bu ikisini karıştırmak sık yapılan bir hatadır.
- vii. Bir metot, hiçbir nesne oluşturulmadan sadece sınıf adıyla çağrılabilirse, bu metot türde bir metottur.
- viii. Java’da bir sabit değer tanımlamak için anahtar kelimesi kullanılır.
- ix. String sınıfında, bir String’in belirtilen indeksteki karakterini döndürmek için metodu kullanılır.
- x. Bir sınıftan yeni bir nesne oluşturulduğunda o sınıfın otomatik çağrılan özel metoduna denir.

YANITLAR

//1. SORU (4 FONKSİYONDAN HER BİRİ 7 PUAN)

```
public void kitapEkle(Kitap k) {
    kitaplar[sayac++]=k;
    if(sayac==kitaplar.length)
        diziyiBuyut();
}

private void diziyiBuyut() {
    Kitap[] kitaplar2=new Kitap[2*kitaplar.length];
    for(int i=0;i<sayac;i++)
        kitaplar2[i]=kitaplar[i];
    kitaplar=kitaplar2;
}

public int kitapSil(int id) {
    int donecek=-1;
    for(int i=0;i<sayac;i++) {
        if(kitaplar[i]!=null && kitaplar[i].getID()==id) {
            donecek=i;
            for(int j=i;j<sayac-1;j++)
                kitaplar[j]=kitaplar[j+1];
            // BU FOR DÖNGÜSÜ PUANLAMAYA DAHİL DEĞİLDİR
            sayac--;
            kitaplar[sayac]=null;
            break;
        }
    }
}
```

```

        }
    }
    return donecek;
}

public void listele() {
    for(Kitap k:kitaplar) {
        if(k!=null)
            System.out.println(k.toString());
    }
}

```

//2. SORU

class Ogrenci{

```

    private int ogrenciNo;
    private String ad;
    private int[] notlar;
    private static int notSayisi;

```

2 PUAN

```

    public Ogrenci(int ogrenciNo,String ad) {
        this.ogrenciNo=ogrenciNo;
        this.ad=ad;
        notlar=new int[5];
    }

```

2 PUAN

```

    public boolean notEkle(int not) {
        if(notSayisi<notlar.length) {
            notlar[notSayisi++]=not;
            return true;
        }
        else
            return false;
    }

```

4 PUAN

```

    public double ortalamaHesapla() {
        double ortalama=0.0;
        for(int x:notlar)
            ortalama+=x;
        return ortalama/notSayisi;
    }

```

4 PUAN

```

    public int getOgrenciNo() {
        return ogrenciNo;
    }

```

2 PUAN

```

    public String getAd() {
        return ad;
    }

```

2 PUAN

```

    public int enYuksekNot() {
        int ey=0;
        for(int i=0;i<notSayisi;i++)
            if(notlar[i]>ey)
                ey=notlar[i];
        return ey;
    }

```

4 PUAN

}

//4.SORU

```
String temizle(String str) {  
    return str.trim();  
}
```

3 PUAN

```
String temizle(String str,char c) {  
    String donecek="";  
    for(int i=0;i<str.length();i++)  
        if(str.charAt(i)!=c)  
            donecek+=str.charAt(i);  
    return donecek;  
}
```

4 PUAN

```
String temizle(String str,boolean sadeceHarfler) {  
    String donecek="";  
    for(int i=0;i<str.length();i++) {  
        char x=str.charAt(i);  
        boolean a_zmi= x>='A' && x<='Z' || x>='a' && x<='z';  
        if(sadeceHarfler && !a_zmi || !sadeceHarfler && a_zmi)  
            donecek+=str.charAt(i);  
    }  
    return donecek;  
}
```

5 PUAN

//5. SORU (HER BİR ŞIK 2 PUAN)

- i. equals veya compareTo
- ii. default access veya package access
- iii. Arrays-sort
- iv. ragged array veya düzensiz dizi
- v. metin havuzu veya string pool
- vi. length-length()
- vii. static
- viii. final
- ix. charAt()
- x. konstrüktör